

SECURITY DESIGN DOCUMENT

By: Krystian Bochenko & Ethan Nayak

A. Architecture Overview

A.1 System Architecture

SecureWebApp is a three-tier web application. The client tier consists of HTML5 templates rendered by Flask's Jinja2 engine. The application tier is a Python/Flask server that handles all HTTP requests, enforces authentication and authorization, validates inputs, and manages file operations. The data layer uses JSON files for structured data and binary .enc files for encrypted document storage. No external database is used, as required by the project specification.

All traffic flows through app.py. Every route is protected by one or more security decorators, so no business logic runs before authentication and authorization checks are complete.

Client Tier

The client tier has ten Jinja2 HTML templates: login.html, register.html, dashboard.html, documents.html, upload.html, edit.html, share.html, admin.html, and error.html. Jinja2 auto-escaping is on by default, so all template variables are HTML-escaped before rendering. This provides baseline XSS protection at the output layer.

Application Tier

The application tier is built in app.py and three supporting modules. app.py defines all Flask routes, validation functions, session management, and RBAC decorators. config.py holds all security constants such as timeouts, limits, and allowed file types. security_logger.py handles structured JSON audit logging. encryption.py wraps the Fernet cipher for file encryption and decryption.

Data Layer

File	Contains	Classification
data/users.json	Username, email addresses, bcrypt password hashes, role, lockout state	RESTRICTED
data/sessions.json	Session tokens, user IDs, creation and last-activity timestamps	RESTRICTED
data/documents.json	Document metadata: owner, sharing permissions, version history	CONFIDENTIAL
uploads/<uuid>.enc	Fernet-encrypted file contents (AES-128-CBC + HMAC-SHA256)	CONFIDENTIAL
logs/security.log	JSON-structured security events with timestamps and user IDs	INTERNAL
data/secret.key	Fernet symmetric encryption key (32 bytes, URL-safe base64)	RESTRICTED — excluded from git

A.2 Data Flow Diagrams

Document Upload Flow

- User submits a multipart form via POST /documents/upload containing a file attachment.
- The @require_auth decorator checks the session cookie. If expired or missing, the user is redirected to /login.
- The file extension is checked against ALLOWED_EXTENSIONS {pdf, txt, docx, png, jpg}. The MIME type is checked against ALLOWED_MIMES separately. Both must pass.
- Raw file bytes are checked against MAX_FILE_SIZE (16MB). The filename is sanitized with Werkzeug's secure_filename() to strip path separators and null bytes.
- encrypt_file() wraps the bytes using Fernet, producing authenticated ciphertext written to uploads/<uuid>.enc.
- A metadata entry is added to documents.json with the document ID, filename, owner, sharing permissions, upload timestamp, and version history.
- A DOCUMENT_UPLOADED event is written to security.log with the filename, document ID, and user ID.

Document Download Flow

- User sends GET /documents/download/<doc_id>.
- @require_auth validates the session and loads the user record from users.json.
- Authorization check: request continues only if the user is the document owner, is listed in shared_with, or has the admin role. All other requests return 403 and log an ACCESS_DENIED event.
- The encrypted file is read from uploads/<doc_id>.enc. decrypt_file() verifies the HMAC and decrypts the file. Tampering causes an InvalidToken error rather than returning bad data.
- Flask's send_file() streams the decrypted bytes to the user as a download. The decrypted content is never written to disk.
- A DOCUMENT_DOWNLOADED event is logged with the document ID and filename.

A.3 Technology Stack Justification

Component	Version	Justification
Python / Flask	3.1.3	Lightweight WSGI framework with a small attack surface. Jinja2 auto-escaping provides baseline XSS protection. Recommended by the project spec.
bcrypt	5.0.0	Adaptive password hashing. Cost factor 12 means each hash takes about 300ms, slowing brute-force attacks. Passwords are salted automatically.
cryptography (Fernet)	46.0.5	Fernet uses AES-128-CBC for encryption and HMAC-SHA256 for integrity. Prevents both unauthorized decryption and undetected file tampering.

flask-limiter	Latest	IP-based rate limiting integrated into Flask's request lifecycle. Applied to /register and /login to prevent brute-force and mass account creation.
Werkzeug	3.1.7	secure_filename() strips path separators, null bytes, and leading dots from uploaded filenames, preventing path traversal attacks.
JSON file storage	N/A	Required by the project spec. Eliminates the SQL injection vulnerability class entirely. Tradeoff is no atomic transactions, which is acceptable at this scale.

B. Threat Model

B.1 Asset Identification

The following assets need protection within SecureWebApp:

Asset	Classification	Impact of Compromise
Fernet secret key (data/secret.key)	RESTRICTED	All encrypted documents become readable. Full confidentiality loss of all stored files.
Password hashes (users.json)	RESTRICTED	Offline cracking attacks become possible. Account takeover for any user whose hash is recovered.
Session tokens (sessions.json)	RESTRICTED	Session hijacking. Attacker gains full access as the victim without knowing their password.
Encrypted documents (.enc files)	CONFIDENTIAL	Contains private user content. Without the Fernet key the files are unreadable, but stealing both enables full decryption.
Document metadata (documents.json)	CONFIDENTIAL	Reveals ownership, sharing relationships, and version history. Modification could escalate access permissions.
Security audit log (security.log)	INTERNAL	Tampering or deleting the log could hide attacker activity and block incident response.

B.2 Threat Enumeration (STRIDE)

Threats are listed using the STRIDE framework: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

Category	Threat Example	Control in Place
Spoofing	Attacker uses a stolen session token to log in as another user	HttpOnly cookies block JS access. SameSite=Strict blocks CSRF. 30-min session timeout limits the exposure window.

Tampering	Attacker edits documents.json to add themselves to shared_with	All write operations go through server-side authorization checks. Direct file access requires server filesystem access, which is out of scope.
Repudiation	User denies having deleted a document or changed a role	Every sensitive action is written to the JSON security log with a timestamp and user ID.
Info Disclosure	Path traversal via filename to read arbitrary files	secure_filename() sanitizes all filenames. The download route resolves paths from a UUID only, never from user input.
Denial of Service	Attacker submits 5 wrong passwords per account to lock everyone out	Account lockout is per-account. Rate limiting on /login and /register blocks rapid automated attempts. Admin can reset locked accounts.
Elev. of Privilege	Regular user navigates to /admin to access user management	@require_role('admin') is applied to all admin routes. Unauthorized access returns 403 and logs an ACCESS_DENIED event.

B.3 Vulnerability Assessment and Attack Scenarios

The following vulnerabilities were identified during self-assessment and testing. All High-severity issues identified during development were addressed before the final penetration test. The penetration test found one Medium and two Low severity issues, documented below.

MEDIUM-01: Debug Mode Enabled in Development

The Flask application was configured with debug=True in the main block. In Flask debug mode, any unhandled exception triggers an interactive Werkzeug debugger in the browser. This would allow arbitrary Python code execution in the server context if encountered in a production environment. During testing, the Werkzeug debugger was visible on error pages.

Remediation: Set debug=False and use os.environ.get('FLASK_DEBUG', False) to control via environment variable. Never deploy with debug mode enabled.

LOW-01: Self-Signed TLS Certificate

The application uses a self-signed TLS certificate. Browsers warn users that the connection is not trusted. Users may get used to ignoring these warnings, which makes it harder to spot real man-in-the-middle attacks in a future deployment.

Remediation: For any production deployment, use a certificate from a trusted Certificate Authority such as Let's Encrypt. Self-signed certificates are acceptable for development.

LOW-02: No Password Reset Functionality

The application does not have a password reset feature. Users who forget their password cannot recover their account without direct admin help. This is a low security risk but affects usability and does not scale well as more users are added.

Remediation: Add a secure password reset flow using time-limited tokens generated with `secrets.token_urlsafe(32)`, sent to the user's registered email address with a 15-minute expiration.

B.4 Risk Prioritization

Finding	Likelihood	Impact	Priority
Debug mode enabled (debug=True)	High	High	MEDIUM
Self-signed TLS certificate	Low	Low	LOW
No password reset functionality	Low	Low	LOW
Concurrent sessions allowed (no single-session limit)	Low	Low	INFO
Deprecation warnings in test suite	Low	Low	INFO
python-dotenv not in requirements.txt	Low	Low	INFO

C. Security Controls

For each control, this section covers: description, implementation, testing methodology, known limitations, and mitigation strategies.

C.1 Authentication

Control: Password-based authentication using bcrypt hashing, complexity enforcement, account lockout, and rate limiting.

Implementation: Registration checks username (3 to 20 alphanumeric characters), email format, and password complexity using `validate_username()`, `validate_email()`, and `validate_password()`. Passwords are hashed with `bcrypt.hashpw()` using `gensalt(rounds=12)` and the decoded string is saved to `users.json`. Plaintext passwords are never stored or logged. Login checks the `locked_until` timestamp, then calls `bcrypt.checkpw()` for constant-time comparison. At 5 failed attempts, the account is locked for 15 minutes. Rate limiting is applied to both `/register` (5 per minute) and `/login` (10 per minute) via `flask-limiter`.

Testing: Registration was tested with passwords missing each complexity requirement. Each returned the correct specific error. Account lockout was tested by submitting 5 wrong passwords. The `security.log` confirms `ACCOUNT_LOCKED` events for users `kb1245235`, `Mom123`, and `admin`. Subsequent attempts during the lockout window returned `LOGIN_BLOCKED` events. Automated testing confirmed 11 tests passed.

Known Limitation: No automated password expiration or forced rotation is implemented. Users may keep the same password indefinitely.

Mitigation: Add a password age policy and prompt users to update passwords after a set period for any production deployment.

C.2 Access Control (RBAC)

Control: Role-based access control with three roles: admin, user, and guest. Enforced via Python decorator functions on every route.

Implementation: Two decorator factories are defined using `functools.wraps`. `require_auth()` checks for a valid session cookie on every protected route. `require_role(role)` also checks that the current user's role matches the required one, returning 403 and logging `ACCESS_DENIED` if not. All `/admin` routes use both decorators. Document operations check ownership and role inline.

Permission Matrix: Admins have full system access including user management and all document operations. Users can upload, edit, delete, and share their own documents, and download documents shared with them. Guests can only view and download documents explicitly shared with them.

Testing: Non-admin users attempting to access `/admin` received 403 responses and `ACCESS_DENIED` log entries. Guest users attempting to upload were blocked at the route level.

Known Limitation: There is no document-level audit trail for edit actions beyond the log entry. The version history tracks who uploaded each version but does not store a diff of changes.

Mitigation: For production, store a checksum or change summary alongside each version entry in `documents.json`.

C.3 Input Validation and Injection Prevention

Control: Whitelist-based input validation, HTML escaping, safe filename handling, and two-layer file type verification.

XSS Prevention: `sanitize_input()` calls `html.escape()` on all form-submitted strings before use. Jinja2 auto-escaping is active on all templates so template variables are always HTML-encoded at render time. The Content-Security-Policy header adds a third layer by restricting script sources to 'self'.

File Upload Validation: Uploads go through two independent checks. First, the file extension is checked against `ALLOWED_EXTENSIONS = {'pdf', 'txt', 'docx', 'png', 'jpg'}`. Second, the MIME type is checked against `ALLOWED_MIMES`. Both must pass. `secure_filename()` is applied before any disk operation. File size is capped at 16MB.

Command and Path Injection: No shell commands are used anywhere in the codebase. All file access uses Python's `open()` with paths built from UUIDs, never from user-supplied strings.

Testing: XSS payloads in username and login fields produced no script execution. Path traversal attempts in the download URL returned 404 responses. Files with invalid extensions and MIME types were rejected with error messages.

Known Limitation: There is no antivirus or malware scanning on uploaded files. The MIME check and extension whitelist reduce risk but do not detect malicious file contents.

Mitigation: For production, integrate a file scanning tool such as ClamAV before writing uploaded files to disk.

C.4 Encryption

At-Rest Encryption: All uploaded files are encrypted with Fernet before writing to disk. Fernet uses AES-128-CBC for confidentiality and HMAC-SHA256 for integrity. The key is generated once via `Fernet.generate_key()` and saved to `data/secret.key`, which is excluded from version control. If the HMAC check fails due to tampering, Fernet raises an `InvalidToken` exception instead of returning corrupted data.

In-Transit Encryption: The Flask application runs with `ssl_context=('cert.pem', 'key.pem')`, enabling HTTPS on port 5000. The certificate and private key are excluded from the repository. A self-signed certificate is used for development.

Password Hashing: `bcrypt` is used for password storage. Passwords cannot be recovered from stored hashes. `bcrypt.checkpw()` uses constant-time comparison to prevent timing side-channel attacks.

Testing: Uploaded files were confirmed to be unreadable in their raw `.enc` form. Decryption succeeded for authorized users and failed for unauthorized access attempts. HTTPS was confirmed active on all routes.

Known Limitation: The Fernet key has no automated rotation. If the key is stolen, all stored files become readable. Rotating the key would require re-encrypting all stored documents.

Mitigation: Implement versioned key management and a rotation schedule for any production deployment.

C.5 Session Management

Control: Cryptographically random session tokens with timeout enforcement and secure cookie configuration.

Implementation: `create_session()` generates a token using `secrets.token_urlsafe(32)`, producing 256 bits of cryptographically secure randomness. The token is stored in `sessions.json` with `user_id`, `created_at`, and `last_activity`. `validate_session()` runs on every request via `@app.before_request`, checking that `last_activity` is within `SESSION_TIMEOUT` (1800 seconds). The session cookie uses `httponly=True`, `samesite='Strict'`, and `secure=True`.

Testing: Session timeout was verified by reducing the timeout value and confirming expired sessions were automatically deleted. Cookie flags were confirmed using browser DevTools.

Known Limitation: Multiple active sessions are allowed per user with no single-session enforcement. A user logged in on two devices simultaneously has two valid sessions.

Mitigation: Add a session listing UI and allow users to view and revoke active sessions, or enforce single-session login if the security policy requires it.

C.6 Security Headers

All responses pass through a `@app.after_request` hook that sets the following headers:

Header	Status	Value and Purpose
Content-Security-Policy	Pass	default-src 'self' — blocks external scripts, frames, and connections
X-Frame-Options	Pass	DENY — prevents clickjacking via iframe embedding
X-Content-Type-Options	Pass	nosniff — prevents MIME-type sniffing attacks
X-XSS-Protection	Pass	1; mode=block — legacy browser XSS filter
Referrer-Policy	Pass	strict-origin-when-cross-origin
Permissions-Policy	Pass	geolocation=(), microphone=(), camera=()
Strict-Transport-Security	Pass	All 7 required headers verified present using browser developer tools

Known Limitation: All seven headers were verified using browser DevTools. HSTS is recommended for any public-facing deployment but is less critical in a development environment with a self-signed certificate.

C.7 Logging and Monitoring

Control: Structured JSON security event logging covering authentication, authorization, data access, and anomaly events.

Implementation: The SecurityLogger class in security_logger.py wraps Python's standard logging module with a custom log_event() method. Each event is a JSON object with a UTC timestamp, event_type, user_id, and a details dictionary. Severity levels map to Python's logging levels: INFO for normal events, WARNING for failed attempts and access denials, and ERROR for account lockouts.

Events logged: USER_REGISTERED, LOGIN_SUCCESS, LOGIN_FAILED, LOGIN_BLOCKED, ACCOUNT_LOCKED, SESSION_CREATED, SESSION_DESTROYED, SESSION_EXPIRED, ACCESS_DENIED, VALIDATION_FAILED, DOCUMENT_UPLOADED, DOCUMENT_DOWNLOADED, DOCUMENT_SHARED, DOCUMENT_EDITED, DOCUMENT_DELETED, USER_DELETED, and ROLE_CHANGED.

Testing: The security.log from live testing confirms all event types are captured. Account lockout sequences were visible for example we both made users with the names of kb1245235 and EthanNayak, then used automated testing and confirmed logging works correctly with 11 passing tests. This showed logging worked and that users could get a lockout if getting their password wrong many times.

Known Limitation: The log file has no tamper protection. A user with filesystem access could modify or delete it. There is no automated alerting for unusual patterns such as repeated failed logins.

Mitigation: For production, write logs to an append-only location or a remote logging service and add monitoring rules for suspicious activity. We wanted to preserve security but also show functionality, so it might look a little different for this project than production would be.

D. Data Protection

D.1 Data Classification

Data in SecureWebApp is grouped into four tiers based on sensitivity and the impact of unauthorized access:

Tier	Assets	Handling Requirements
RESTRICTED	Fernet key, password hashes, session tokens	Never logged or transmitted in cleartext. Excluded from version control. Access limited to the application process only.
CONFIDENTIAL	Encrypted document files, document metadata, user email addresses	Files stored encrypted. Transmitted only over HTTPS. Access gated by RBAC.
INTERNAL	Security audit logs, session timestamps, failed attempt counts	Server-side only. Not exposed to end users. Required for incident response and audit trail.
PUBLIC	Username (used in sharing UI), document filenames, role labels	Rendered in the UI. Still subject to XSS escaping and input validation.

D.2 Encryption Methods

File Encryption (Fernet): Fernet is a symmetric authenticated encryption scheme from the cryptography library. It uses AES-128-CBC for encryption and HMAC-SHA256 for authentication. Fernet tokens include the initialization vector, ciphertext, and HMAC in a single URL-safe base64-encoded blob. Authentication is always checked before decryption, so any modification to the ciphertext causes an InvalidToken error instead of returning bad data. This protects both confidentiality and integrity.

Password Hashing (bcrypt): bcrypt is a password-hashing function based on the Blowfish cipher. It is intentionally slow. At cost factor 12, each hash takes about 300ms on modern hardware, making offline brute-force attacks slow for strong passwords. Each hash includes a random salt, which prevents rainbow table attacks.

Transport Encryption (TLS): Flask runs with an SSL context pointing to a self-signed X.509 certificate. This encrypts all traffic between the client browser and the server, including session cookies and file contents in transit.

D.3 Key Management

Generation: The Fernet key is generated on first run using `Fernet.generate_key()`, which calls `os.urandom(32)` internally to produce 32 bytes of cryptographically secure random data encoded as URL-safe base64.

Storage: The key is saved to `data/secret.key` as raw bytes. This file is excluded from version control via `.gitignore`. The `get_or_create_key()` function loads it on each encrypt or decrypt operation, or generates a new one if absent.

Known Limitation: There is no automated key rotation. If the Fernet key is stolen, all encrypted files become readable. Re-encryption of all stored documents would be needed to recover. A production system would use versioned keys and a rotation schedule.

D.4 Secure Deletion Procedures

Document Deletion: When a document is deleted via `/documents/delete/<doc_id>`, the server calls `os.remove()` on the `.enc` file, then removes the metadata entry from `documents.json`. The deletion is logged as a `DOCUMENT_DELETED` event. Only the document owner or an admin can delete.

Limitation: `os.remove()` marks the file as deleted in the filesystem but does not overwrite the raw bytes. On a standard filesystem, data may remain recoverable until those disk sectors are reused. A secure delete utility such as `shred` on Linux would be more thorough for sensitive production data.

Session Cleanup: `destroy_session()` removes the session entry from `sessions.json` on logout. `validate_session()` deletes expired sessions on their next access. This stops stale session tokens from staying valid indefinitely.

User Deletion: Admin deletion via `/admin/delete_user/<username>` removes the user record from `users.json` but does not delete or reassign documents owned by that user. Their `.enc` files remain on disk. A production system would handle this with a cascading delete or an ownership transfer process.